

Mini-Rapport d'Analyse Statique

OWASP UnCrackable Level 1

Auteur Chagdaly Hiba
Date 02 Mai 2026
Cours Sécurité des applications mobiles
Lab Lab 04 – JADX GUI + dex2jar + JD-GUI
APK OWASP UnCrackable-Level1.apk

*Ce rapport est réalisé dans un cadre pédagogique.
L'application analysée est volontairement vulnérable (OWASP MASTG).*

Informations générales sur l'application

Propriété	Valeur
Package	owasp.mstg.uncrackable1
Version	1.0
minSdkVersion	19 (Android 4.4 KitKat)
targetSdkVersion	28 (Android 9 Pie)
SHA-256	1da8bf57d266109f9a07c01bf7111a1975ce01f190b9d914bcd3ae3dbef96f21
Source	OWASP MASTG – application pédagogique volontairement vulnérable

TABLE 1 – Informations de l'APK analysé

```
mobexler@Mobexler ~
└─> ls APK-Analysis
UnCrackable-Level1.apk
mobexler@Mobexler ~
└─> |
```

FIGURE 1 – APK dans le dossier de travail

```
mobexler@Mobexler ~/APK-Analysis
└─> hexdump -n 4 UnCrackable-Level1.apk
00000000 4b50 0403
00000004
mobexler@Mobexler ~/APK-Analysis
└─> |
```

FIGURE 2 – Vérification signature ZIP (PK)

```
mobexler@Mobexler ~/APK-Analysis
└─> sha256sum UnCrackable-Level1.apk
1da8bf57d266109f9a07c01bf7111a1975ce01f190b9d914bcd3ae3dbef96f21  UnCrackable-Level1.apk
mobexler@Mobexler ~/APK-Analysis
└─> |
```

FIGURE 3 – Hash SHA-256 de l'APK

Permissions et composants exposés

2.1 Permissions déclarées

Aucune permission Android n'est déclarée dans le manifeste de cette application.

2.2 Composants exportés

Composant	Type	Raison	Risque
MainActivity	Activity	Intent-filter exported="false"	sans [MOYENNE]

TABLE 2 – Composants exportés

Note : Depuis Android 12, tout composant avec un intent-filter doit définir explicitement l'attribut `exported`. L'absence de `exported="false"` rend MainActivity accessible depuis d'autres applications.

2.3 Configuration du manifeste

Attribut	Valeur	Risque
android:allowBackup	true	[MOYENNE]
android:debuggable	Non défini	Faible
android:usesCleartextTraffic	Non défini	Faible

TABLE 3 – Attributs sensibles du manifeste

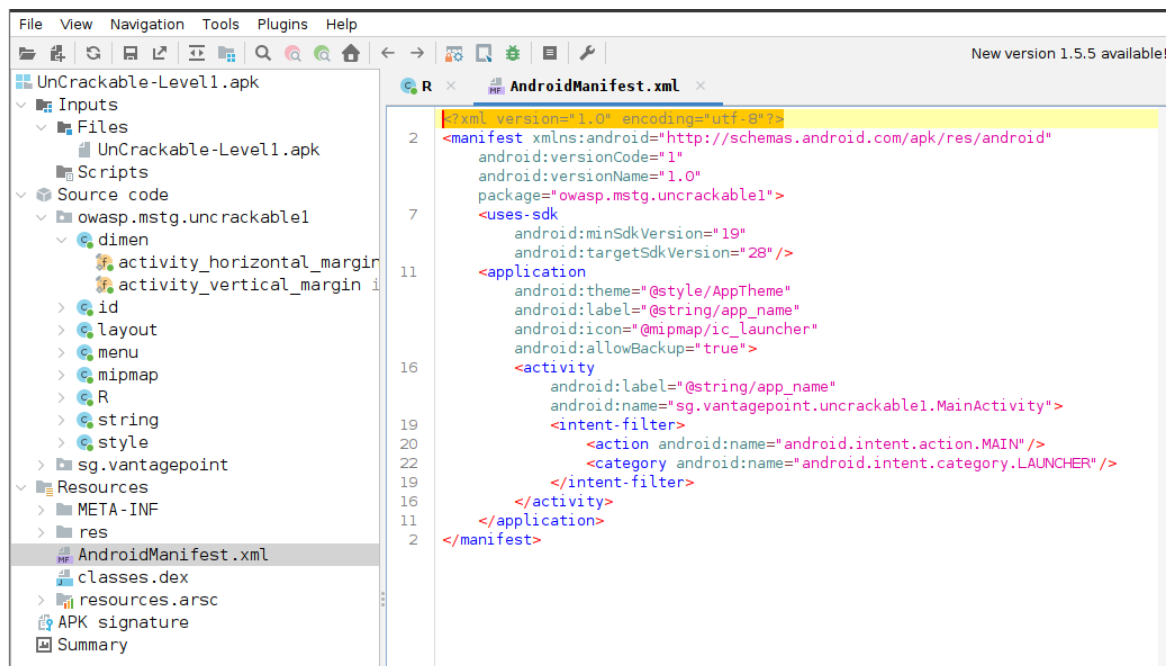


FIGURE 4 – AndroidManifest.xml analysé dans JADX GUI

Constats de sécurité

Constat 1 – Clé AES codée en dur [ELEVÉE]

Localisation : sg.vantagepoint.uncrackable1.a, méthode a(String str)

Observation : La clé de chiffrement AES est hardcodée directement dans le code source sous forme de chaîne hexadécimale :

```
bArr = sg.vantagepoint.a.a.a(
    b("8d127684cbc37c17616d806cf50473cc"), // Cle AES hardcodee
    Base64.decode("5UJiFctbmgbDoLXmpL12mkno8HT4Lv8dlat8FxR2G0c=", 0)
);
```

```
/* loaded from: classes.dex */
public class a {
    public static boolean a(String str) {
        byte[] bArr;
        byte[] bArr2 = new byte[0];
        try {
            bArr = sg.vantagepoint.a.a.a(b("8d127684cbc37c17616d806cf50473cc"), Base64.decode("5UJiFctbmgbDoLXmpL12mkno8HT4L
        }) catch (Exception e) {
            Log.d("CodeCheck", "AES error:" + e.getMessage());
            bArr = bArr2;
        }
        return str.equals(new String(bArr));
    }

    public static byte[] b(String str) {
        int length = str.length();
        byte[] bArr = new byte[length / 2];
        for (int i = 0; i < length; i += 2) {
            bArr[i / 2] = (byte) ((Character.digit(str.charAt(i), 16) << 4) + Character.digit(str.charAt(i + 1), 16));
        }
        return bArr;
    }
}
```

FIGURE 5 – Clé AES et secret chiffré dans JADX

Impact : Tout attaquant ayant accès à l'APK peut extraire la clé et déchiffrer le secret sans exécuter l'application.

Remédiation :

- Ne jamais stocker de clés cryptographiques dans le code source
- Utiliser Android Keystore System pour stocker les clés de manière sécurisée
- Dériver les clés à partir d'une entrée utilisateur avec PBKDF2

Constat 2 – Algorithme AES/ECB (mode non sécurisé) [ELEVÉE]

Localisation : sg.vantagepoint.a, méthode a(byte[], byte[])

Observation : L'algorithme utilisé est AES en mode ECB :

```
SecretKeySpec secretKeySpec = new SecretKeySpec(bArr, "AES/ECB/PKCS7Padding"
);
Cipher cipher = Cipher.getInstance("AES");
```

```
public class a {  
    public static byte[] a(byte[] bArr, byte[] bArr2) {  
        SecretKeySpec secretKeySpec = new SecretKeySpec(bArr, "AES/ECB/PKCS7Padding");  
        Cipher cipher = Cipher.getInstance("AES");  
        cipher.init(2, secretKeySpec);  
        return cipher.doFinal(bArr2);  
    }  
}
```

FIGURE 6 – Classe AES – mode ECB utilisé

Impact : Le mode ECB est déterministe – un même bloc de texte clair produit toujours le même bloc chiffré. Il est vulnérable aux attaques par analyse de motifs et ne doit jamais être utilisé pour des données sensibles.

Remédiation :

- Remplacer AES/ECB par AES/GCM qui offre un chiffrement authentifié
- Utiliser un vecteur d'initialisation (IV) aléatoire à chaque chiffrement

Constat 3 – Secret stocké dans le binaire [ELEVÉE]

Localisation : sg.vantagepoint.uncrackable1.a

Observation : Le secret attendu est stocké chiffré en Base64 dans le code. Combiné à la clé hardcodée, le déchiffrement est trivial :

```
import base64
from Crypto.Cipher import AES

key = bytes.fromhex('8d127684cbc37c17616d806cf50473cc')
encrypted = base64.b64decode('5UJiFctbmgBDoLXmpL12mkno8HT4Lv8dlat8FxR2G0c=')
cipher = AES.new(key, AES.MODE_ECB)
print(cipher.decrypt(encrypted))
# Resultat : b'I want to believe\x0f\x0f...'
```

```
mobexler@Mobexler ~/APK-Analysis  
└─ python3 -c "  
import base64  
from Crypto.Cipher import AES  
key = bytes.fromhex('8d127684cbc37c17616d806cf50473cc')  
encrypted = base64.b64decode('5UJiFctbmgbDoLXmpLl2mknoHT4Lv8dlat8FxR2G0c=')  
cipher = AES.new(key, AES.MODE_ECB)  
print(cipher.decrypt(encrypted))  
"  
b'I want to believe\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f'
```

FIGURE 7 – Secret déchiffré : "I want to believe"

Impact : N'importe quel utilisateur peut retrouver le secret sans même lancer l'application, rendant la protection totalement inefficace.

Remédiation :

- Ne jamais stocker de secrets dans le binaire, même chiffrés
- Implémenter la vérification côté serveur via une API sécurisée
- Utiliser des protocoles à divulgation nulle de connaissance (zero-knowledge)

Constat 4 – android:allowBackup="true" [MOYENNE]

Localisation : AndroidManifest.xml

Observation : L'attribut android:allowBackup="true" est défini, ce qui permet l'extraction des données via ADB :

```
adb backup -noapk owasp.mstg.uncrackable1
```

Impact : Toute personne ayant accès physique à l'appareil peut extraire les SharedPreferences, bases de données et fichiers internes de l'application.

Remédiation :

- Définir android:allowBackup="false" dans le manifeste
- Si la sauvegarde est nécessaire, utiliser android:fullBackupContent avec des règles d'exclusion pour les données sensibles

Constat 5 – Détection root contournable [MOYENNE]

Localisation : sg.vantagepoint.c (méthodes a, b, c)

Observation : Trois méthodes de détection root sont implémentées mais toutes contournables par hooking (Frida/Objectection) :

```
// c.a() : cherche le binaire su dans le PATH
for (String str : System.getenv("PATH").split(":")) {
    if (new File(str, "su").exists()) return true;
}
// c.b() : verifie Build.TAGS contient "test-keys"
return str != null && str.contains("test-keys");
// c.c() : cherche Superuser.apk, daemonsu, etc.
```

```
/* loaded from: classes.dex */
public class c {
    public static boolean a() {
        for (String str : System.getenv("PATH").split(":")) {
            if (new File(str, "su").exists()) {
                return true;
            }
        }
        return false;
    }

    public static boolean b() {
        String str = Build.TAGS;
        return str != null && str.contains("test-keys");
    }

    public static boolean c() {
        for (String str : new String[]{"/system/app/Superuser.apk", "/system/xbin/daemonsu", "/system/etc/init.d/99SuperSUDaemon", "/system/bin/.ext/.su", "/system/et
        if (new File(str).exists()) {
            return true;
        }
    }
    return false;
}
}
```

FIGURE 8 – Classe c – 3 méthodes de détection root

Remédiation :

- Utiliser des bibliothèques natives (.so) pour les vérifications de sécurité
- Implémenter SafetyNet Attestation / Play Integrity API côté serveur

Comparaison des outils JADX GUI vs JD-GUI

Aspect	JADX GUI	JD-GUI
Accès manifeste	Oui, directement	Non
Ressources XML	Oui	Non
Code Android	Excellent	Bon
Gestion Kotlin	Bonne	Limitée
Obfuscation	Tente reconstruction	Conserve noms obfusqués
Usage	Analyse APK complète	Vérification JAR/bytecode

TABLE 4 – Comparaison JADX GUI vs JD-GUI

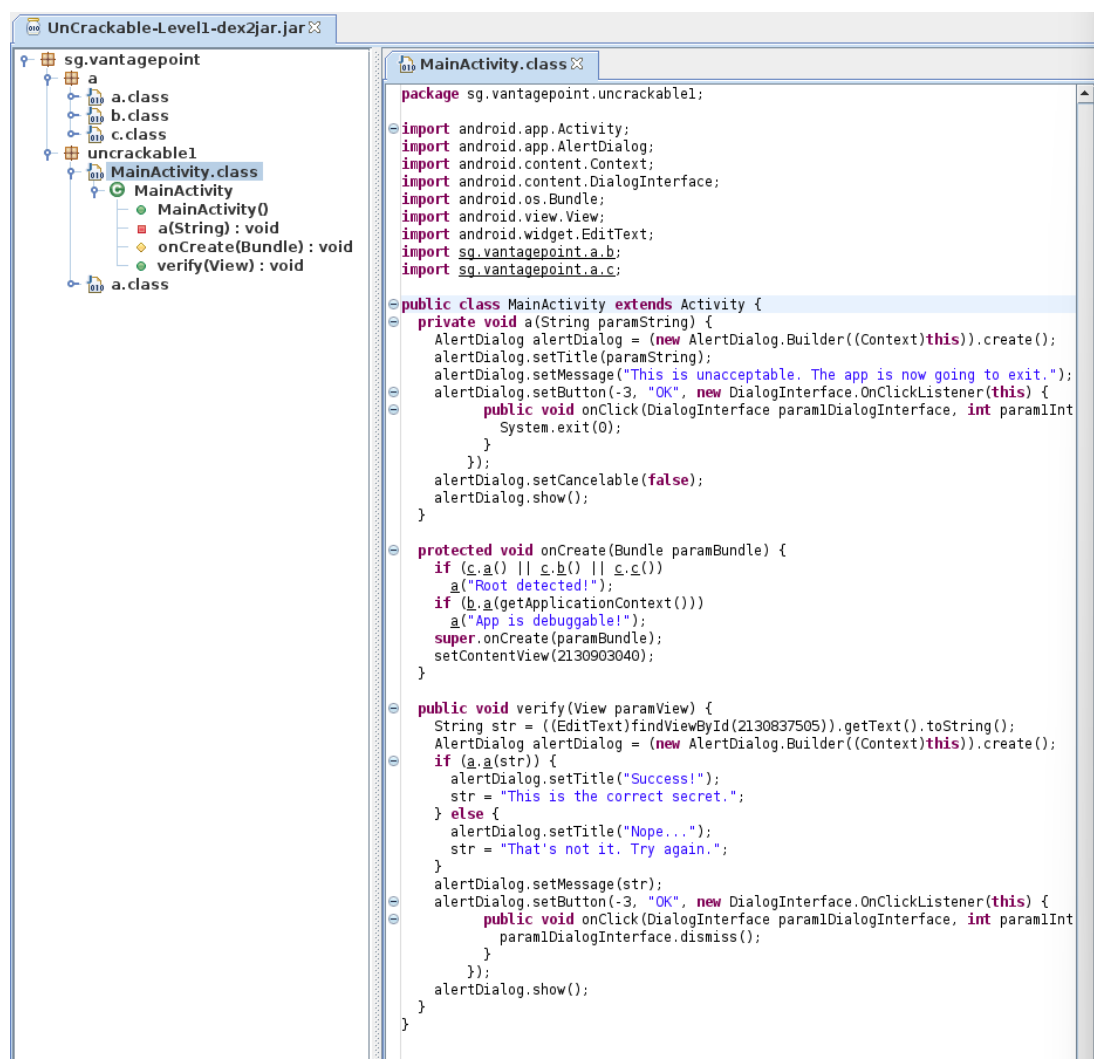


FIGURE 9 – Analyse de MainActivity.class dans JD-GUI

```

mobexler@Mobexler ~/APK-Analysis
└─> ls
UnCrackable-Level1.apk  UnCrackable-Level1-dex2jar.jar
mobexler@Mobexler ~/APK-Analysis

```

FIGURE 10 – Conversion DEX vers JAR avec dex2jar

Synthèse des risques

Constat	Sévérité	Remédiation prioritaire
Clé AES hardcodée	[ELEVÉE]	Android Keystore
AES/ECB non sécurisé	[ELEVÉE]	Migrer vers AES/GCM
Secret dans le binaire	[ELEVÉE]	Vérification serveur
allowBackup=true	[MOYENNE]	allowBackup=false
Détection root contournable	[MOYENNE]	Play Integrity API

TABLE 5 – Tableau de synthèse des vulnérabilités

Conclusion

L'analyse statique de UnCrackable Level 1 illustre les erreurs classiques de sécurité mobile : stockage de secrets dans le code, algorithme cryptographique faible (AES/ECB), et protections contournables statiquement. Le secret **"I want to believe"** a été retrouvé en quelques minutes sans exécuter l'application.

Ces vulnérabilités auraient un impact critique dans une application réelle, permettant à un attaquant d'extraire des clés, identifiants ou secrets sans aucun accès réseau.